❀ Flower Shop E-Commerce Project Plan

Project idea

Let's imagine a flower shop called **Bloom & Blossom**.

Customers should be able to:

1. Browse flowers
2. View flower details
3. Add flowers to a cart
4. Change quantities
5. Remove flowers
6. See the total price
7. Place an order
8. Enter their delivery information
9. Receive an order confirmation

FULL APPLICATION ROAD MAP

- ➕ Add products
- 📝 Update products
- 🗑️ Remove products
- 🔍 Search products
- 💰 Track prices
- 🎁 Track stock
- 📊 Calculate totals
- 👥 Manage customers
- 💾 Save everything with `localStorage`
- ⏰ Use dates/timers
- 🔄 Load data asynchronously with `Promises`
- 🌐 Eventually communicate with a backend using HTTP
- 📦 Organize the code using JavaScript modules
- 📄 Later connect it to a **Node.js backend**

🚀 Our development order

I strongly recommend that we don't try to build everything at once.

We'll go:

```
1. Project setup
       ↓
2. Homepage
       ↓
3. Product data
       ↓
4. Product cards
       ↓
5. Search/filter
       ↓
6. Product details
       ↓
7. Add to cart
       ↓
8. Cart
       ↓
9. Update/remove cart items
       ↓
10. localStorage
       ↓
11. Checkout
       ↓
12. Orders
       ↓
13. Admin dashboard
       ↓
14. Modules
       ↓
15. Promises / async
       ↓
16. Node.js
       ↓
17. HTTP/API
       ↓
18. Database
       ↓
19. Authentication
       ↓
20. Deployment
```

Phase 1 — Project foundation

Goal

Create the basic structure of the website.

```
flower-shop/
│
├── index.html
├── shop.html
├── product.html
├── cart.html
├── checkout.html
├── order-success.html
│
├── css/
│   ├── style.css
│   ├── shop.css
│   ├── cart.css
│   └── checkout.css
│
├── js/
│   ├── main.js
│   ├── products.js
│   ├── cart.js
│   ├── checkout.js
│   └── utils.js
│
└── images/
```

Phase 2 — Homepage

Build a professional landing page.

Sections

```
Navbar
   ↓
Hero section
   ↓
Featured flowers
   ↓
Flower categories
   ↓
Why choose us?
   ↓
Customer testimonials
   ↓
Call to action
   ↓
Footer
```

Example categories:

- Roses
- Bouquets
- Lilies
- Wedding flowers
- Birthday flowers
- Gift arrangements

Phase 3 — Product data

This is where your JavaScript learning becomes important.

Instead of hardcoding every flower into HTML, we'll create product objects.

For example:

```
const products = [
    {
        id: 1,
        name: "Red Rose Bouquet",
        price: 45000,
        category: "Roses",
        stock: 10,
        image: "images/red-roses.jpg"
    },

    {
        id: 2,
        name: "White Lily Bouquet",
        price: 55000,
        category: "Lilies",
        stock: 7,
        image: "images/white-lilies.jpg"
    }
];
```

🛵 □ Phase 4 — Product listing

Create the shop page.

```
              FLOWERS

[ Search flowers... ]

[All] [Roses] [Lilies] [Bouquets]

┌──────────┐  ┌──────────┐  ┌──────────┐
│  Flower  │  │  Flower  │  │  Flower  │
│  image   │  │  image   │  │  image   │
│  Rose    │  │  Lily    │  │  Bouquet │
│ 45,000   │  │ 55,000   │  │ 70,000   │
│ [Add]    │  │ [Add]    │  │ [Add]    │
└──────────┘  └──────────┘  └──────────┘
```

JavaScript concepts

We'll practice:

- `forEach()`
- `map()`
- `filter()`
- `find()`
- Functions
- Event listeners
- DOM manipulation

🔎 Phase 5 — Search and filtering

Customers should be able to search.

For example:

```
Search: rose
```

JavaScript finds matching products.

We'll use:

```
filter()
```

We'll also allow:

```
All flowers
Roses
Lilies
Bouquets
Wedding
Birthday
```

❁ Phase 6 — Product details

When the customer clicks a flower:

```
Red Rose Bouquet

[       IMAGE       ]

Price: UGX 45,000

Beautiful arrangement of fresh red roses.

Available: 10

Quantity: [-] 1 [+]

[ Add to Cart ]
```

We'll learn how to:

- Get a product by ID
- Display dynamic information
- Handle URL parameters
- Work with objects
- Handle user input

🛒 Phase 7 — Shopping cart

This is one of the most important parts.

Customer clicks:

```
Add to Cart
```

JavaScript adds the product to the cart.

Example:

```
SHOPPING CART

Red Rose Bouquet
UGX 45,000

Quantity: [-] 2 [+]

Subtotal: UGX 90,000

White Lily Bouquet
UGX 55,000

Quantity: [-] 1 [+]

Subtotal: UGX 55,000

-------------------------
Total: UGX 145,000

[Proceed to Checkout]
```

JavaScript concepts

We'll use:

- Arrays
- Objects
- `find()`

- `filter()`
- `reduce()`
- Functions
- Events

Especially:

`reduce()` for calculating the total.

🗃 Phase 8 — LocalStorage

Now we make the application remember the customer's cart.

Without `localStorage`:

```
Refresh page
     ↓
Cart disappears ✗
```

With `localStorage`:

```
Refresh page
     ↓
Cart remains ✓
```

You'll practice exactly what you've already been learning:

`JSON.stringify()`

and

`JSON.parse()`

This will make your previous localStorage exercises much more meaningful.

Phase 9 — Cart management

Customers should be able to:

Increase quantity

```
[ + ]
```

Decrease quantity

```
[ - ]
```

Remove product

```
[ Remove ]
```

Clear entire cart

```
[ Clear Cart ]
```

And importantly:

The page must update immediately.

That's directly connected to the problem you were solving with your fruit project.

💳 Phase 10 — Checkout

Create a checkout form.

```
CHECKOUT

Customer Information

Name
[_________________]

Phone
[_________________]

Email
[_________________]

Delivery Address
[_________________]

Delivery Date
[_________________]

Payment Method

○ Mobile Money
○ Cash on Delivery

-------------------------

Order Total:
UGX 145,000

[ Place Order ]
```

We'll practice:

- Forms
- Input validation
- Events

- Objects
- Functions
- Conditional logic

🎁 Phase 11 — Order creation

When the customer clicks:

```
Place Order
```

we create an order object.

Something like:

```
const order = {
    id: 1001,
    customer: {
        name: "...",
        phone: "...",
        address: "..."
    },
    items: [...],
    total: 145000,
    status: "Pending"
};
```

Now we're starting to think like backend developers.

💐 Phase 12 — Order confirmation

Display something like:

```
ORDER CONFIRMED ✿

Thank you for your order!

Order #: FLW1001

Total: UGX 145,000

Your flowers will be delivered to:

Gulu, Uganda

[ Continue Shopping ]
```

Phase 13 — Admin dashboard

Once the customer side works, we can build a simple admin area.

```
ADMIN DASHBOARD

Products       25
Orders         18
Revenue        UGX 2,450,000
Low Stock       4
```

Admin can:

```
Add product
Edit product
Delete product
View orders
Update order status
```

This will bring us back to many of the things you've already practiced.

□ Phase 14 — Advanced JavaScript

Once the frontend is working, we introduce more advanced concepts.

Modules

```
import { products } from "./products.js";
```

Promises

For operations that take time.

`async/await`

For cleaner asynchronous code.

HTTP

We'll learn:

```
Frontend
    ↓
HTTP request
    ↓
Backend
    ↓
Database
```

Phase 15 — Node.js backend

This is where your current learning path eventually leads.

Instead of:

```
JavaScript
     ↓
localStorage
```

we move toward:

```
               INTERNET
                  ↓
              FRONTEND
                  ↓
                HTTP
                  ↓
              NODE.JS
                  ↓
               DATABASE
```

The backend could eventually handle:

- Products
- Customers
- Orders
- Authentication
- Payments
- Stock
- Order status

🔐 Phase 16 — Authentication

Eventually:

```
LOGIN

Email
[_____________]

Password
[_____________]

[ Login ]
```

And:

```
Customer
    ↓
Customer dashboard
```

while:

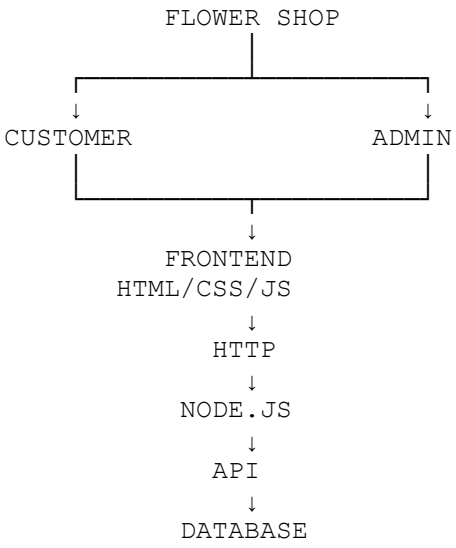
```
Admin
    ↓
Admin dashboard
```

This should come **much later**, after we understand the basic application.

🗄 □ Phase 17 — Database

Eventually replace our temporary `localStorage` data with a real database.

Our architecture becomes:



HTML STRUCTURE.

CSS STRUCTURE

JAVASCRIPT STRUCTURE

What you're learning	Where we can use it
Arrays	Products/customers
Objects	Product information
Functions	Add/update/remove logic
`if/else`	Stock and validation
Loops	Displaying records
`map()`	Transforming data
`filter()`	Searching/filtering
`find()`	Finding a specific product
`reduce()`	Calculating totals
DOM	Updating the webpage
Events	Buttons/forms
`localStorage`	Saving data
JSON	Storing/retrieving data
Modules	Organizing the project
Promises	Async operations
HTTP	Communicating with a server
Node.js	Backend
Classes/OOP	Structuring application logic